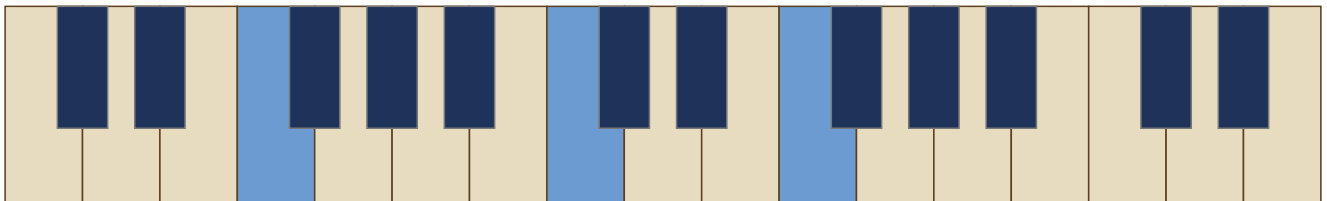


Ivory

Rust Rewrite -- Project Guide

~/Dropbox/Ivory-Rust

A MIDI keyboard monitor and chord detector, rebuilt from Python / PySide6 into Rust. The GUI uses egui (immediate-mode, pure Rust) and MIDI input goes through midir / CoreMIDI. This document covers the project layout, how each file works, key Rust patterns illustrated along the way, and the chord-detection algorithm in plain language.



Keyboard rendered by piano_widget.rs using egui::Painter

Contents

1. Quick Start	2
2. Project Layout	2
3. Architecture & Data Flow	3
4. File-by-File Rust Guide	4
5. The Chord Detection Algorithm	5
6. Tests & Known Bugs	6
7. Next Steps	7
8. Dependency Reference	7

1. Quick Start

Prerequisites: the Rust toolchain (rustup.rs) and a USB/Bluetooth MIDI keyboard.

```
# Launch the app (debug build -- shows the piano window)
cargo run -p ivory

# Optimised release build (~5x faster)
cargo run -p ivory --release

# Run unit tests for the chord-detection library
cargo test -p ivory-core
```

Right-click anywhere in the piano window to open the settings menu. Options include dark mode, flat/sharp name preference, keytoggle mode (click piano keys with the mouse to latch notes), and the debug panel (shows the top 5 candidate chords with scores).

2. Project Layout

```
Ivory-Rust/
|
+-- Cargo.toml      Workspace definition -- lists both crates
+-- README.md       Project overview and quick-start
+-- .gitignore
|
+-- ivory-core/      Pure library (no GUI, fully testable)
|   +-- Cargo.toml
|   +-- src/
|       +-- lib.rs    Re-exports ChordDetector
|       +-- patterns.rs All chord/scale data tables
|       +-- detector.rs ChordDetector struct + 42 unit tests
|
+-- ivory/           Binary (GUI + MIDI)
    +-- Cargo.toml
    +-- src/
        +-- main.rs    Entry point -- window title, initial size
        +-- app.rs      IvoryApp: egui update loop, MIDI polling
        +-- piano_widget.rs Custom 88-key piano renderer
        +-- midi.rs     Background MIDI thread (CoreMIDI on macOS)
        +-- settings.rs Persistent JSON config
```

The split into two crates is intentional. `ivory-core` contains all music-theory logic with zero external dependencies. You can test it in isolation, benchmark it, or reuse it from a different front-end without touching the GUI.

3. Architecture & Data Flow

A background OS thread reads raw MIDI bytes from CoreMIDI via `midir` and sends parsed `MidiEvent` values down an `mpsc` channel. The `egui` update loop drains that channel on every frame, updates the active notes, calls `ChordDetector`, then draws everything.

```
CoreMIDI
  | raw bytes
  v
midi.rs  parse_and_send()
  | MidiEvent (mpsc::channel)
  v
app.rs   process_midi() --> active_notes: HashMap<u8, ()>
  |
  v
update_chord() --> ivory_core::ChordDetector::detect_chord()
  | Option<String>
  v
egui Painter --> chord label text + coloured piano keys
```

Cargo workspace

The root `Cargo.toml` declares a workspace with two members. ``cargo build`` in the root compiles both; ``cargo build -p ivory-core`` compiles only the library. Both crates share a single `Cargo.lock`, so dependency versions are consistent.

Settings persistence

`IvoryApp` loads settings at startup from `~/.config/ivory/settings.json` and saves on request. The `Settings` struct uses `#[derive(Serialize, Deserialize)]` so `serde_json` handles all the JSON work automatically.

```
#[derive(Serialize, Deserialize)]
pub struct Settings {
    pub dark_mode: bool,
    pub prefer_flats: bool,
    pub white_key_active_color: Color,
    // ...
}
```

4. File-by-File Rust Guide

File	Key Rust concepts
patterns.rs	static slices, match as zero-cost lookup, 'static lifetime
detector.rs	HashSet operations, Iterator chains, Option<T>, labeled loops ('outer:')
midi.rs	mpsc channel, move closures, RAIL guard (_connection field)
app.rs	&mut self in event loop, HashMap<K,V>, immediate-mode GUI pattern
piano_widget.rs	egui::Painter, Rect arithmetic, pure functions with no state
settings.rs	serde derive macros, serde_json, Result<T,E>, std::fs

patterns.rs -- static data tables

Python dicts iterate in insertion order (Python 3.7+) and that order controls detection priority. The Rust version uses a static slice of tuples to preserve that ordering at zero runtime cost:

```
pub static CHORD_PATTERNS: &[amp;str, amp;u8]] = amp;[
    ("major",      amp;[0, 4, 7]),
    ("minor",      amp;[0, 3, 7]),
    ("diminished", amp;[0, 3, 6]),
    // ... 100+ entries in priority order
];
```

Essential and optional intervals are match functions -- a Rust match compiles to a jump table and costs nothing at runtime:

```
pub fn essential_for(chord_type: amp;str) -> amp;'static [u8] {
    match chord_type {
        "major7" => amp;[4, 11],    // M3 + M7
        "minor7" => amp;[3, 10],    // m3 + m7
        _        => amp;[],
    }
}
```

midi.rs -- RAIL connection guard

midir requires a closure that lives as long as the MIDI connection. The connection is stored as _connection in MidiThread -- the underscore prefix tells Rust not to warn about it being 'unused'. When MidiThread is dropped the connection closes automatically (RAIL).

```
pub struct MidiThread {
    _connection: midir::MidiInputConnection<()>, // kept alive here
}

let conn = input.connect(
    amp;port, "ivory-in",
    move |_stamp, message, _| { // 'move' transfers tx into closure
        parse_and_send(message, amp;&tx);
    },
    (),
).ok()?;
```

piano_widget.rs -- custom rendering

The piano is drawn entirely with egui::Painter. There is no widget state -- draw_piano() is a pure function that reads active_notes and settings and emits rectangles and lines. The 88 keys span A0 (MIDI 21) to C8 (108). Black keys are 70% as wide as white keys and 65% as tall.

5. The Chord Detection Algorithm

The algorithm is a scoring engine: for every set of active notes it generates all plausible chord interpretations and picks the highest score.

Step 1 -- Normalise to pitch classes

MIDI note numbers (0-127) are folded mod 12 to get pitch classes (0=C, 1=C#/Db, ... 11=B). Octave information is discarded for chord matching.

Step 2 -- Early-return special cases

Several patterns are detected before the main loop to avoid ambiguity:

- 2 notes -> interval name (C (P5), C (M3), ...)
- 4-note pattern [0,1,7,10] from bass -> minor 6 slash chord (e.g. Bbm6/C)
- 5-note dim7 upper structure -> dominant 7b9 (C E G Bb Db -> C7(b9))
- Half-diminished vs minor 6 (Gm7b5 / Bbm6 enharmonic disambiguation)

Step 3 -- Score every root + pattern combination

For each pitch class as a candidate root, intervals from that root are computed and scored against every entry in CHORD_PATTERNS.

```
score = essential_score      // 0-60: are the 3rd and 7th present?
+ pct_match                  // 0-40: % of input notes matched
+ highest_note_bonus         // +10 if highest note is in pattern
+ completeness_bonus         // +30 for exact match, 0 otherwise
+ rootless_bonus             // +15 if root omitted but 3rd+7th present
+ root_in_bass_bonus         // +15 if root == lowest note
+ inversion_bonus            // +35/40 if bass is a chord tone
+ special_bonus              // context-specific adjustments
- extra_penalty              // -3 per note outside the pattern
- missing_penalty            // -40 essential, -1 optional, -8 other
```

Step 4 -- Post-loop overrides

After scoring, a few rules adjust or override the winner:

- Diminished and augmented chords are symmetrical -- always use the bass as root.
- Slash chords: if root != bass, try simplifying the upper chord (C7/E -> C/E).
- Guard: don't simplify a 3-note triad when only 2 notes remain after removing bass.

Step 5 -- Scale fallback

If 5 or more notes are clustered within one octave, detect_scale() is tried. A perfect scale match scores 5000+ and will override any chord interpretation. This handles cases like playing all 7 notes of F major -- the result is 'F Ionian' rather than some extended chord.

Special bonuses (selected)

Situation	Bonus	Reason
minor11 shell perfect match	+8000	beats scale detection (6000)
6/9 with root in bass, exact match	+9000	beats m11 (8000)
maj7(6/9) exact match	+10000	beats 6/9 (9000)
half-dim7 perfect match	+700	beats minor slash chords
dim major7 perfect match	+500	rare chord, high confidence
m6 slash [0,1,7,10] from bass	+1500	specific voicing pattern

6. Tests & Known Bugs

Running the tests

```
cargo test -p ivory-core          # 33 pass, 9 fail
cargo test -p ivory-core -- --nocapture # show printed output
cargo test -p ivory-core t_major    # run one test by name
```

Tests live at the bottom of `detector.rs` inside a `#[cfg(test)]` module.

```
#[cfg(test)]
mod tests {
    use super::*;
    fn notes(v: &[u8]) -> HashSet<u8> { v.iter().copied().collect() }

    #[test]
    fn t_major() {
        assert_eq!(
            ChordDetector::new().detect_chord(&notes(&[60, 64, 67])),
            Some("C".into())
        );
    }
}
```

9 known failing tests (mirror Python bugs)

Test	Input notes	Expected	Got
t_cm7_not_am6	C Eb G Bb	Cm7	Eb6
t_min7	C Eb G Bb	Cm7	Eb6
t_minor_inv	Eb3 G3 C4	Cm/Eb	Eb6
t_7sus4	C F G Bb	C7sus4	Gm11
t_9sus	C D F Bb	C9(sus)	Bb(add9)/C
t_7s9	C E G Bb Eb	C7(#9)	E?7(#11)
t_7s11	C E G Bb F#	C7(#11)	Gb7(b9,#11)
t_7b13	C E G Bb Ab	C7(b13)	Bb7(#11)
t_rootless_dom9	E Bb D	C9	E7(#11)

Root cause of the Eb6 group: the minor7-to-6th reinterpretation fires for all minor7 matches when the chord span is < 12 semitones. It should only fire when the m3 of the detected root equals the bass note (the Am7 -> C6 case). The altered-dominant failures all have the 5th present, which causes a competing interpretation from another root to win.

7. Suggested Next Steps

Fix the Eb6/Cm7 bug

Guard the minor7-to-6th reinterpretation in detector.rs. Only apply it when $(\text{root_pc} + 3) \% 12 == \text{lowest_pc}$. This fixes `t_cm7_not_am6`, `t_min7`, and `t_minor_inv` in one change.

Fix altered dominants with 5th

The 7b9/7#9/7b13 failures all have the 5th present. The current scoring lets a major7#11 or similar win from a different root. Add a boost for dominant7 + altered tension when the root is the lowest note.

Add colour pickers

egui has `color_edit_button_rgb()` built in. In `app.rs`, replace the checkbox grid with colour buttons wired to settings fields. Call `settings.save()` on any change.

Release packaging on macOS

`cargo build --release -p ivory` produces a single binary at `target/release/ivory`. Wrap it in an `.app` bundle with a simple shell script or `cargo-bundle` (`cargo install cargo-bundle`).

Port the remaining 48 Python tests

The Python suite has 90 tests; Rust has 42. Adding the missing ones will surface more edge cases and give you clear goals to fix.

Explore ownership via the debug panel

The debug panel (right-click -> Debug mode) shows top-5 candidates. Try extending it to highlight which intervals caused each candidate -- good exercise in passing data from `ivory-core` up to the GUI.

8. Dependency Reference

Crate	Ver	Used in	Purpose
egui	0.29	ivory	Immediate-mode GUI widgets and rendering
eframe	0.29	ivory	OS window creation and main loop
midir	0.9	ivory	MIDI input via CoreMIDI (macOS)
serde	1	ivory	Derive Serialize / Deserialize for Settings
serde_json	1	ivory	Read/write settings.json
dirs-next	2	ivory	Locate <code>~/.config/ivory/</code> reliably
(none)	--	ivory-core	Zero external deps by design

Useful cargo commands

```
cargo check           # fast type-check, no binary produced
cargo fmt             # auto-format all source files
cargo clippy          # run lints (great for learning idioms)
cargo doc --open -p ivory-core # view generated API docs in browser
cargo update          # update deps to latest compatible
RUSTFLAGS="-D warnings" cargo build # treat warnings as errors
```